CE-321L/CS-330L: Computer Architecture

RISC V Pipelined Processor

Aashir Ahmed, M. Ammar Maqdoom, M.Sufyan Siddiqui

21 April 2025

Introduction

This project aimed to design and implement a 5-stage pipelined RISC-V processor capable of executing a bubble sort algorithm on a predefined array of integers. The project was divided into three main tasks:

1. Converting bubble sort pseudocode into RISC-V assembly code, verifying it on the Venus simulator, and adapting a single-cycle processor to execute the sorting program.
2. Transforming the single-cycle processor into a pipelined architecture and testing individual instructions to ensure correct functionality.
3. Incorporating hazard detection circuitry to handle data, control, and structural hazards through forwarding, stalling, and pipeline flushing.
4. Comparing the performance of the single-cycle and pipelined processors in terms of execution time for the bubble sort program.

Methodology

Task 1:

1.1 Bubble Sort Pseudocode to Machine Code

The first task involved implementing a bubble sort algorithm in RISC-V assembly, verified using the Venus simulator. The algorithm sorts an array of seven 4-byte integers stored in data memory starting at address 0x100. The pseudocode was adapted from prior lab exercises, with modifications to support a 64-bit processor architecture. The assembly code, embedded in the instruction memory, initializes registers, sets up the array, and executes nested loops to compare and swap adjacent elements. Key instructions include:

- `addi` for register initialization (e.g., setting loop counters and array base address).
- `slli` for calculating 4-byte offsets (e.g., `slli x7, x9, 2` for i*4).
- `lw` and `sw` for loading and storing array elements.
- `beq` and `blt` for loop control and conditional branching.

The array was initialized with values [5, 2, 1, 7, 4, 3, 6], and the code was tested on Venus to confirm correct sorting into [1, 2, 3, 4, 5, 6, 7].

Address	+0	+1	+2	+3
0x00000100	5	0	0	0
0x00000104	2	0	0	0

0x00000108	1	0	0	0
0x0000010C	7	0	0	0
0x00000110	4	0	0	0
0x00000114	3	0	0	0
0x00000118	6	0	0	0

Below is the C code of our sorting algorithm:

```
void bubbleSort(int arr[], int size)
{
    int i, j;
    for (i = 0; i < size; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

This is the assembly code implement of the above sorting algorithm:

```
    addi x18, x0, 0       # i = 0 (not used directly in loop)
    addi x19, x0, 7       # n = 7
    addi x8,  x0, 0       # j = 0
    addi x10, x10, 0      # Clear x10 (flag for swap)

    # array initialization moved to Data_Memory module (not shown
here)
```

```
Outer_Loop:
    beq x8, x19, End       # if j == n, end sorting
    addi x9, x0, 0         # i = 0
    addi x10, x0, 0        # reset swap flag
    addi x6, x19, -1       # x6 = n - 1
    sub x6, x6, x8         # x6 = n - 1 - j

Inner_Loop:
    beq x9, x6, After_Inner_Loop  # if i == n - 1 - j, end inner loop
    slli x7, x9, 2         # x7 = i * 4
    addi x7, x7, 256       # x7 = x7 + 256 (array base address)
    lw x5, 0(x7)           # x5 = A[i]

    addi x29, x9, 1        # x29 = i + 1
    slli x28, x29, 2       # x28 = (i + 1) * 4
    addi x28, x28, 256     # x28 = x28 + 256
    lw x30, 0(x28)         # x30 = A[i + 1]

    blt x30, x5, Do_Swap   # if A[i+1] < A[i], swap

    addi x9, x9, 1         # i++
    beq x0, x0, Inner_Loop

Do_Swap:
    mv x31, x5             # temp = A[i]
    mv x5, x30             # A[i] = A[i+1]
    sw x5, 0(x7)           # store A[i]
    mv x30, x31            # A[i+1] = temp
    sw x30, 0(x28)         # store A[i+1]

    addi x10, x0, 1        # set swap flag
    addi x9, x9, 1         # i++
    beq x0, x0, Inner_Loop

After_Inner_Loop:
    addi x8, x8, 1         # j++
    beq x10, x0, End       # if no swap, array is sorted
    beq x0, x0, Outer_Loop
```

```
End:
    # Done
```

1.2 Bubble Sort Implementation Single Cycle

We made a few minor adjustments to the Lab 11 module, where we assembled all the parts to construct the processor. In order to manage branch operations, we added a branch unit and modified the ALU and instruction memory code. To accommodate our 64-bit processor, we changed the 4-byte offset to an 8-byte offset and added support for the funct3 bit of the bgt and blt instructions in the ALU code. Next, we used the instruction memory to initialize a list and sort it on the single-cycle processor. Here are a few sample pieces of code for your review:

ALU 64 bit:

```
module ALU64bit(
    input [63:0] A,
    input [63:0] B,
    input [3:0] ALUOp,
    output reg [63:0] Result
    );

always @(*) begin
    case (ALUOp)
        4'b0000: Result = A & B; //AND
        4'b0001: Result = A | B; //OR
        4'b0010: Result = A + B; //ADD
        4'b0110: Result = A - B; //SUB
        4'b1100: Result = ~(A | B); //NOR
        4'b1000: Result = A * (2 ** B); // SLLI
    endcase

end
endmodule
```

Branch Unit:

```
module Branch_unit(
   input [2:0] Funct3,
   input [63:0] ReadData1,
   input [63:0] ReadData2,
   output reg addermuxselect
  );

  initial
    begin
      addermuxselect = 1'b0;
    end

  always @(*)
    begin
      case (Funct3)
        3'b000:
          begin
            if (ReadData1 == ReadData2)
              addermuxselect = 1'b1;
            else
              addermuxselect = 1'b0;
            end
         3'b100:
            begin
              if (ReadData1 < ReadData2)
              addermuxselect = 1'b1;
            else
              addermuxselect = 1'b0;
            end
        3'b101:
          begin
            if (ReadData1 > ReadData2)
              addermuxselect = 1'b1;
            else
              addermuxselect = 1'b0;
          end
      endcase
```

```
    end
endmodule
```

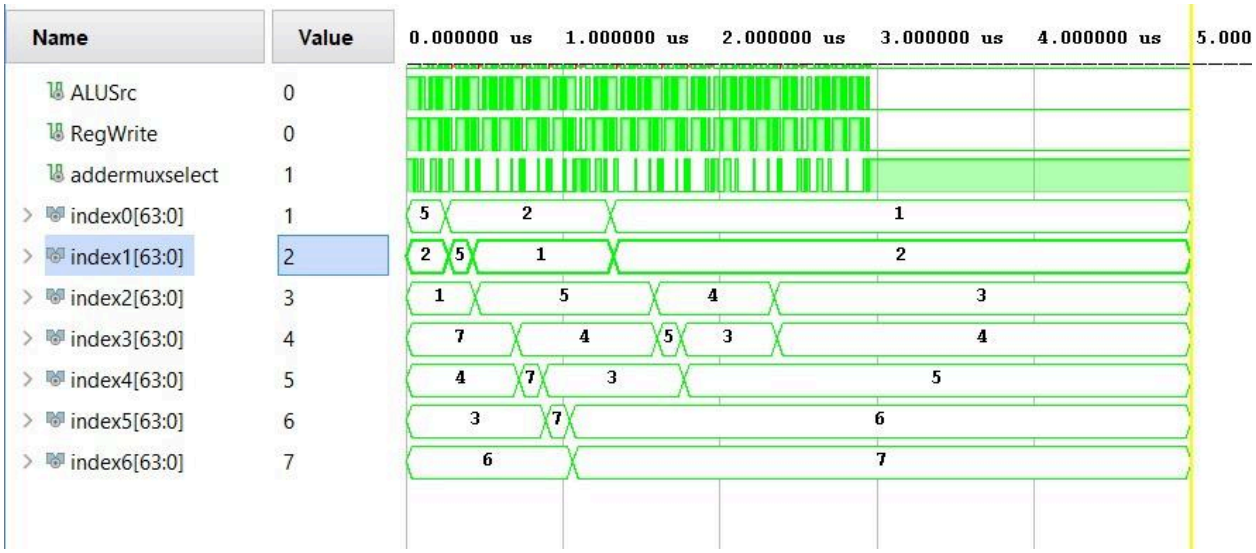
Mux 2x1:

```
module Mux_2x1(
    input [63:0] A,
    input [63:0] B,
    input S,
    output [63:0] Out
    );
    assign Out = (S == 1'b0) ? A : B;
endmodule
```

- **ALU Enhancements:** The ALU was updated to support branch instructions (`bgt`, `blt`) through subtraction (`SUB`) and comparison operations. It performs 64-bit operations (AND, OR, ADD, SUB, NOR, SLLI), controlled by the ALU control unit.
- **Instruction Memory:** The instruction memory was populated with the bubble sort program, using 4-byte offsets for array indexing (e.g., `slli x7, x9, 2`) to align with the 64-bit architecture. Instructions were encoded to support load (`lw`), store (`sw`), and branch operations.
- **Data Memory:** The data memory was initialized with the array at address 0x100, handling 4-byte data sign-extended to 64 bits. Read and write operations were adjusted for 4-byte accesses, ensuring compatibility with the sorting program.
- **Branch Unit:** A new branch unit was added to evaluate conditions for `beq`, `blt`, and `bgt`, setting a multiplexer select signal based on register comparisons. This enabled accurate control flow for the sorting loops.

- **Program Counter and Multiplexers:** The program counter managed instruction address fetching, updated via a multiplexer that selects between PC+4 and branch targets. Additional multiplexers handled ALU operand selection (register vs. immediate) and write-back data (ALU result vs. memory data).
- **Control and Immediate Generation**: The control unit generated signals for R-type, I-type, S-type, and SB-type instructions, with provisions for stall handling. The immediate generator produced sign-extended immediates for load, store, and branch instructions.
- **Register File:** The register file managed 32 64-bit registers, supporting simultaneous reads and writes with reset functionality.
- **Instruction Parsing**: The instruction parser extracted fields (opcode, RD, Funct3, RS1, RS2, Funct7) from 32-bit instructions, enabling accurate decoding.

1.3 Simulation Output:



Task 2:

2.1 Pipelined RISC V Processor

After running the algorithm on the single-cycle processor, we updated the code to add pipelining. To do this, we followed our course book and added special modules that act like storage between each stage of the pipeline:

- IF/ID

```
module IF_ID(
    input clk,
    input reset,
    input [31:0] Instruction,
    input [63:0] PC_Out,
    output reg [31:0] IFID_Instruction,
    output reg [63:0] IFID_PC_Out
    );
always @(posedge clk or posedge reset) begin
    if (reset == 1'b1) begin
        IFID_Instruction = 0; IFID_PC_Out = 0;
    end
    else begin
        IFID_Instruction = Instruction; IFID_PC_Out = PC_Out;
    end
end
endmodule
```

- ID/EX

```
module ID_EX(
    input clk, reset,
    input Branch, MemRead, MemWrite, MemtoReg, RegWrite, ALUSrc,
    input [1:0] ALUOp,
    input [3:0] Funct,
    input [4:0] RS1, RS2, RD,
    input [63:0] IFID_PC_Out, ReadData1, ReadData2, Imm,
    output reg IDEX_Branch, IDEX_MemRead, IDEX_MemWrite, IDEX_MemtoReg,
IDEX_RegWrite, IDEX_ALUSrc,
    output reg [1:0] IDEX_ALUOp,
    output reg [3:0] IDEX_Funct,
    output reg [4:0] IDEX_RS1, IDEX_RS2, IDEX_RD,
    output reg [63:0] IDEX_PC_Out, IDEX_ReadData1, IDEX_ReadData2, IDEX_Imm
);

always @(posedge clk or posedge reset) begin
    if (reset == 1'b1) begin
        IDEX_Branch     <= 0;
        IDEX_MemRead    <= 0;
        IDEX_MemWrite   <= 0;
        IDEX_MemtoReg   <= 0;
        IDEX_RegWrite   <= 0;
        IDEX_ALUSrc     <= 0;
        IDEX_ALUOp      <= 0;
        IDEX_Funct      <= 0;
        IDEX_RS1        <= 0;
        IDEX_RS2        <= 0;
        IDEX_RD         <= 0;
        IDEX_PC_Out     <= 0;
        IDEX_ReadData1  <= 0;
        IDEX_ReadData2  <= 0;
        IDEX_Imm        <= 0;
    end else begin
        IDEX_Branch     <= Branch;
        IDEX_MemRead    <= MemRead;
        IDEX_MemWrite   <= MemWrite;
        IDEX_MemtoReg   <= MemtoReg;
        IDEX_RegWrite   <= RegWrite;
        IDEX_ALUSrc     <= ALUSrc;
        IDEX_ALUOp      <= ALUOp;
```

```
        IDEX_Funct      <= Funct;
        IDEX_RS1        <= RS1;
        IDEX_RS2        <= RS2;
        IDEX_RD         <= RD;
        IDEX_PC_Out     <= IFID_PC_Out;
        IDEX_ReadData1  <= ReadData1;
        IDEX_ReadData2  <= ReadData2;
        IDEX_Imm        <= Imm;
    end
end

endmodule
```

- EX/MEM

```
module EX_MEM(
    input clk, reset,
    input Branch, MemRead, MemWrite, MemtoReg, RegWrite,
    input Zero,
    input [4:0] RD,
    input [63:0] Adder2Out, Result, WriteData,
    output reg EM_Branch, EM_MemRead, EM_MemWrite, EM_MemtoReg,
EM_RegWrite,
    output reg EM_Zero,
    output reg [4:0] EM_RD,
    output reg [63:0] EM_Adder2Out, EM_Result, EM_WriteData
    );
always @(posedge clk or posedge reset) begin
    if (reset == 1'b1) begin
        EM_Branch = 0; EM_MemRead = 0; EM_MemWrite = 0; EM_MemtoReg = 0;
        EM_RegWrite = 0; EM_Zero = 0; EM_RD = 0; EM_Adder2Out = 0;
        EM_Result = 0; EM_WriteData = 0;
    end
    else begin
        EM_Branch = Branch; EM_MemRead = MemRead; EM_MemWrite = MemWrite;
        EM_MemtoReg = MemtoReg; EM_RegWrite = RegWrite; EM_Zero = Zero;
        EM_RD = RD; EM_Adder2Out = Adder2Out; EM_Result = Result;
        EM_WriteData = WriteData;
    end
```

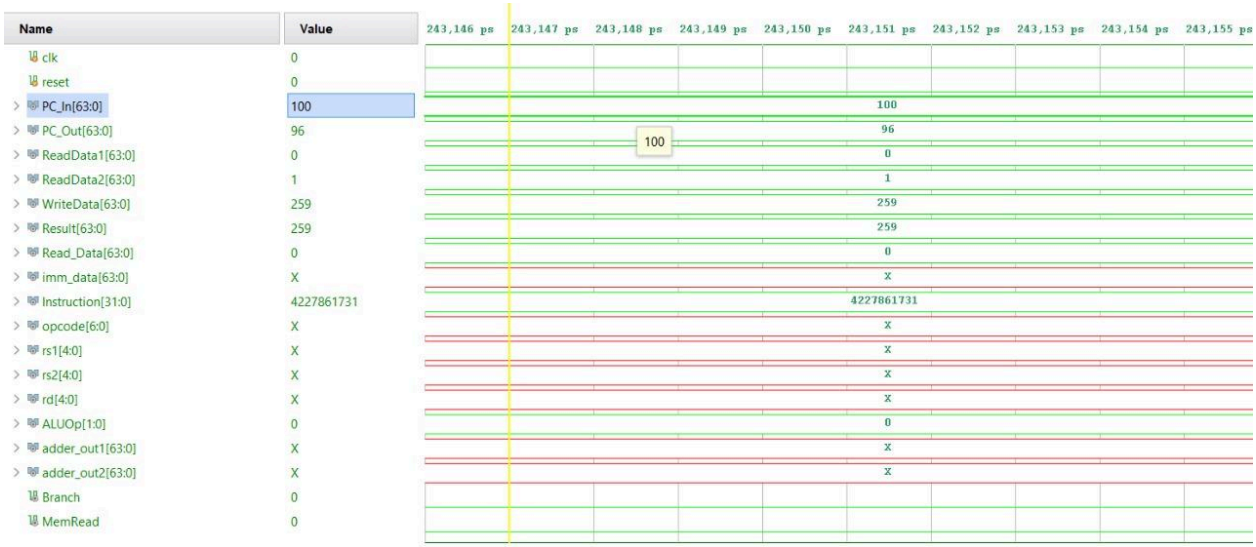
```
end
endmodule
```

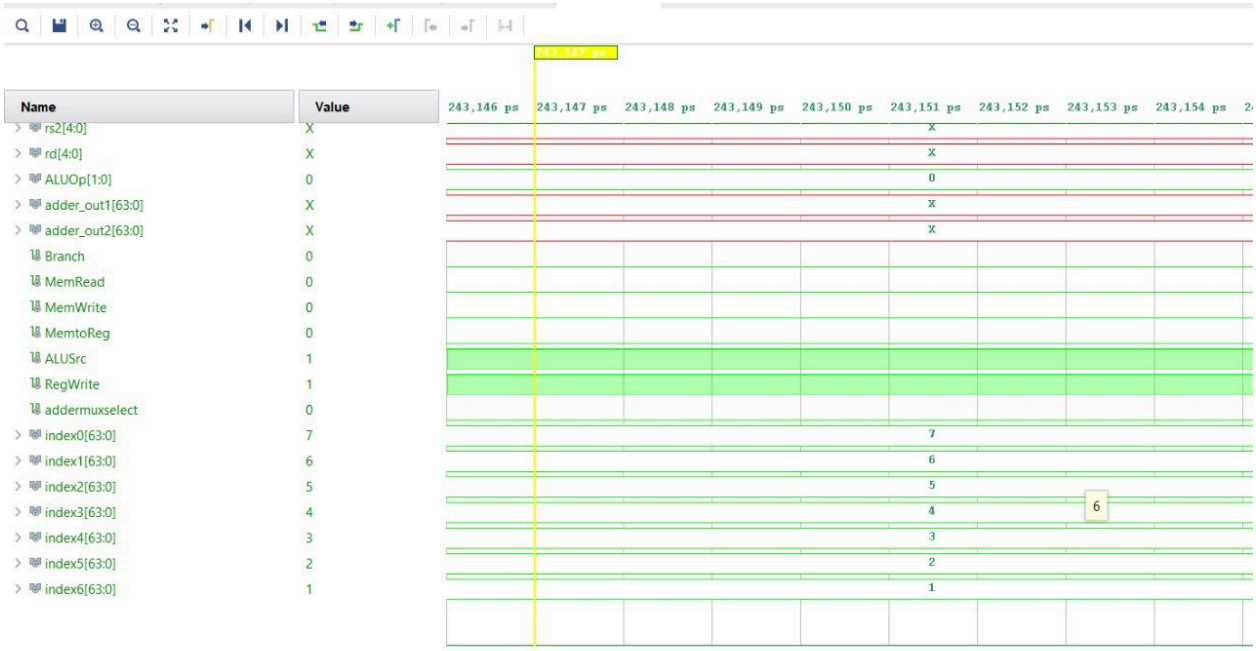
- MEM/WB

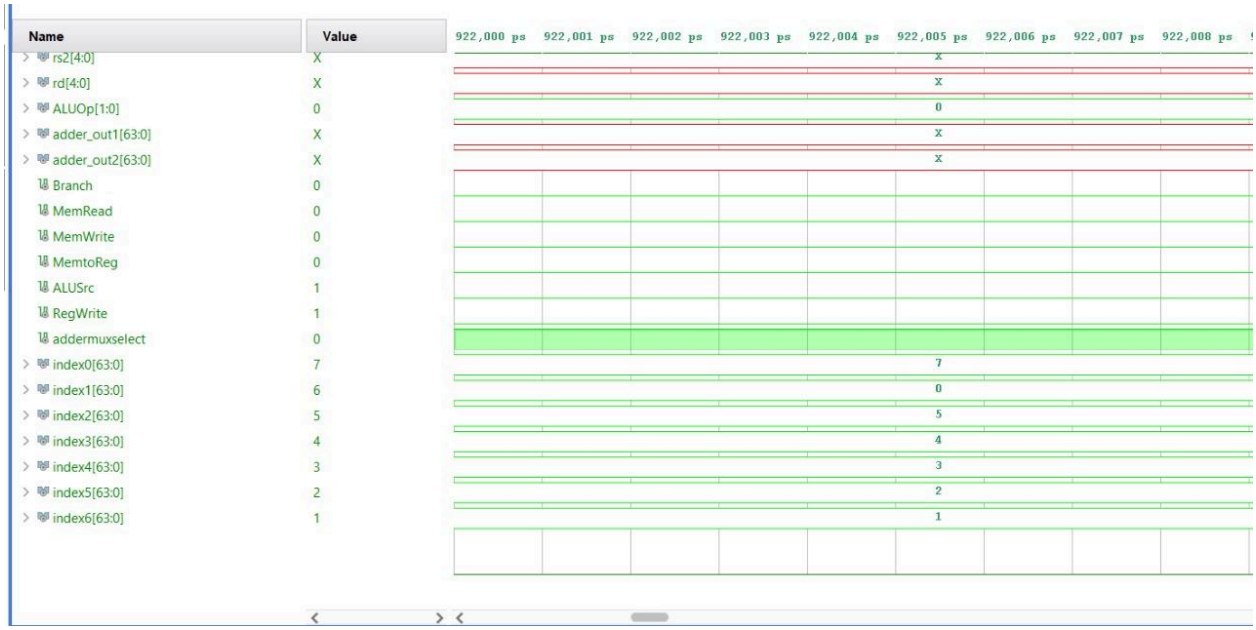
```
module MEM_WB(
    input clk, reset,
    input MemtoReg, RegWrite,
    input [4:0] RD,
    input [63:0] EM_Result, Read_Data,
    output reg MW_MemtoReg, MW_RegWrite,
    output reg [4:0] MW_RD,
    output reg [63:0] MW_Result, MW_Read_Data
    );
always @(posedge clk or posedge reset) begin
    if (reset == 1'b1) begin
        MW_MemtoReg = 0; MW_RegWrite = 0; MW_RD = 0;
        MW_Result = 0; MW_Read_Data = 0;
    end
    else begin
        MW_MemtoReg = MemtoReg; MW_RegWrite = RegWrite; MW_RD = RD;
        MW_Result = EM_Result; MW_Read_Data = Read_Data;
    end
end
endmodule
```

Each instruction's data is stored by these modules before being passed on to the pipeline's subsequent stage. To ensure that everything operated as intended, we tested each instruction individually. By upgrading our earlier code, we also included a forwarding unit, which enables data to move through the pipeline instantly. The hazard detection unit and this forwarding unit collaborate to determine the best time and location for data transmission.

2.2 Simulation Output:






Name
Value
922,000 ps
922,001 ps
922,002 ps
922,003 ps
922,004 ps
922,005 ps
922,006 ps
922,007 ps
922,008 ps
rs2[4:0] X x
rd[4:0] X x
ALUOp[1:0] 0 0
adder_out1[63:0] X x
adder_out2[63:0] X x
Branch 0
MemRead 0
MemWrite 0
MemtoReg 0
ALUSrc 1
RegWrite 1
addermuxselect 0
index0[63:0] 7 7
index1[63:0] 6 0
index2[63:0] 5 5
index3[63:0] 4 4
index4[63:0] 3 3
index5[63:0] 2 2
index6[63:0] 1 1

Task 3:

3.1 Implementing Hazard Detection

By including hazard detection circuitry and, when required, stalls in the pipeline, we addressed data, structural, and control hazards in our code. These risks typically arise from data dependencies or when specific data must be sent. We put in place a hazard detection unit to handle this, which determines when to flush or stall the pipeline and notifies the forwarding unit accordingly.

```
module Hazard_Detection_Unit(
    input [4:0] IF_ID_RS1,
    input [4:0] IF_ID_RS2,
    input [4:0] ID_EX_RD,
    input ID_EX_MemRead,
    input Branch,
    input Jump,
    output reg Stall_Pipeline,
    output reg Flush_Pipeline
);

    // Initial values
    initial begin
        Stall_Pipeline = 1'b0;
        Flush_Pipeline = 1'b0;
    end

    always @(*) begin
        // Load-use hazard detection
        if (ID_EX_MemRead &&
            ((ID_EX_RD == IF_ID_RS1) || (ID_EX_RD == IF_ID_RS2)) &&
            (ID_EX_RD != 5'b00000)) begin
            // Need to stall the pipeline
            Stall_Pipeline = 1'b1;
```

```
        end else begin
            Stall_Pipeline = 1'b0;
        end

        // Branch/Jump hazard detection (to flush the pipeline)
        if (Branch || Jump) begin
            Flush_Pipeline = 1'b1;
        end else begin
            Flush_Pipeline = 1'b0;
        end
    end
endmodule


module Branch_Prediction_Unit(
    input clk,
    input reset,
    input [63:0] PC,
    input Branch_Outcome,      // Actual branch outcome from EX stage
    input Branch_Resolved,     // Signal indicating branch resolution
    output reg Branch_Predicted // Prediction result
);
    // Simple 1-bit branch predictor
    // Using a small table of 16 entries for branch prediction
    reg [15:0] prediction_table;
    wire [3:0] index = PC[5:2]; // Use lower bits of PC as index

    initial begin
        prediction_table = 16'b0; // Initialize to "not taken"
        Branch_Predicted = 1'b0;
    end

    // Make prediction based on history
    always @(*) begin
        Branch_Predicted = prediction_table[index];
    end

    // Update prediction table based on actual outcome
    always @(posedge clk or posedge reset) begin
```

```
        if (reset) begin
            prediction_table <= 16'b0;
        end
        else if (Branch_Resolved) begin
            prediction_table[index] <= Branch_Outcome;
        end
    end
endmodule
```

For Task 3, the code we supplied isn't functioning properly. One major problem was the Program Counter (PC) stalling issue, which caused pipeline disruptions when the PC failed to update properly during stall operations. Maintaining the correct instruction flow was made more difficult by this problem, which necessitated intensive debugging to guarantee stable hazard handling.

Performance Comparison:

In general, pipelined processors are supposed to operate more quickly than non-pipelined ones. But in our instance, the pipelined processor's latency was approximately 6000 ns longer than the non-pipelined version, which was only 3000 ns. This may be because we only finished task 2, which involved pipelining a few instructions. We know that pipelining is not appropriate for a small number of instructions and that it results in more delay than a typical single cycle processor would if our instructions were few, so we avoided stalling, forwarding, and other issues. Although pipelining was successfully implemented, it resulted in a number of inefficiencies and performance problems. We must further examine the current design and make optimization changes in order to get better results.

Conclusion and Challenges:

With modifications to support 4-byte data and branch operations, the project successfully deployed a single-cycle RISC-V processor that could carry out a bubble sort algorithm. Nevertheless, there were a number of difficulties with the implementation. Progress was hampered by Vivado crashes and file compatibility problems across systems, necessitating repeated attempts to stabilize the development environment. Another major obstacle was simulation stability, especially when it came to guaranteeing consistent outcomes. Dependency and control flow problems made the branch unit and hazard detection circuitry implementation difficult, particularly when it came to fixing the Program Counter (PC) stalling issue. Data memory access also presented difficulties because it needed to be carefully configured to handle

4-byte data with appropriate sign-extension and read/write timing in order to guarantee proper operation during the bubble sort execution. Conversion of instruction memory code

Appendix:

https://github.com/Aashir27/RISCV_Pipelined_Processor.git